Pashov Audit Group

Reserve Governor Security Review

February 27th 2026 - March 2nd 2026

Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. About Reserve Governor 4
5. Executive Summary 4
6. Findings 5

Medium findings 7

[M-01] Improper order of operations in `cancelLock` expose funds to theft 7
[M-02] CurrentAccountedNativeRewards uses current balance for reward 7

Low findings 9

[L-01] Eoa check can be bypassed 9
[L-02] Mismatched rounding in ThrottleLib reduces effective proposal capacity 9
[L-03] Uint256 overflow in reward calculations for tokens with high decimals 10
[L-04] ExecuteBatchBypass should be restricted to the governor address 11
[L-05] Incorrect registry event on duplicate selector updates 12
[L-06] Reward token removal without prior accrual loses user reward 13
[L-07] Removing a reward token strand unaccounted balance in vault 14
[L-08] Optimistic and standard proposals collide in shared ID space 15
[L-09] Governor asserts on CLOCK_MODE causing gas exhaustion failure 16
[L-10] OptimisticSelectorRegistry allows zero address governor 16
[L-11] Optimistic proposal state boundaries incorrect due to `>=` comparison 17

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Reserve Governor

Reserve Governor is a hybrid optimistic and standard on-chain governance system for the Reserve protocol, enabling two proposal paths through a single timelock with veto capabilities. It includes an ERC4626 staking vault with vote-locking, multi-token rewards, a selector registry for fast-path whitelisting, and a throttle mechanism limiting optimistic proposal frequency.

5. Executive Summary

A time-boxed security review of the **reserve-protocol/reserve-governor** repository was done by Pashov Audit Group, during which **unforgiven**, **ast3ros**, **Hunter**, **rokinot** engaged to review **Reserve Governor**. A total of **13** issues were uncovered.

Protocol Summary	
Project Name	Reserve Governor
Protocol Type	Governance system
Timeline	February 27th 2026 - March 2nd 2026

Review commit hash:

- 2c0467eecf69562a8e7e94bb7a2ac6a9d11b86ac (reserve-protocol/reserve-governor)

Fixes review commit hash:

- 0d13d691e12f762215f2c856a5037894c2e2bf95 (reserve-protocol/reserve-governor)

Scope

`Deployer.sol` `OptimisticSelectorRegistryDeployer.sol` `ProposalLibDeployer.sol` `ReserveOptimisticGovernorDeployer.sol` `ReserveOptimisticGovernorDeployerDeployer.sol` `StakingVaultDeployer.sol` `ThrottleLibDeployer.sol` `TimelockControllerOptimisticDeployer.sol` `OptimisticSelectorRegistry.sol` `ReserveOptimisticGovernor.sol` `TimelockControllerOptimistic.sol` `ProposalLib.sol` `ThrottleLib.sol` `IDeployer.sol` `IOptimisticSelectorRegistry.sol` `IReserveOptimisticGovernor.sol` `ITimelockControllerOptimistic.sol` `StakingVault.sol` `UnstakingManager.sol` `Constants.sol` `Versioned.sol`

6. Findings

Findings count

Severity	Amount
Medium	2
Low	11
Total findings	**13**

Summary of findings

ID	Title	Severity	Status
[M-01]	Improper order of operations in `cancelLock` expose funds to theft	Medium	**Resolved**
[M-02]	CurrentAccountedNativeRewards uses current balance for reward	Medium	**Resolved**
[L-01]	Eoa check can be bypassed	Low	Acknowledged
[L-02]	Mismatched rounding in ThrottleLib reduces effective proposal capacity	Low	**Resolved**
[L-03]	Uint256 overflow in reward calculations for tokens with high decimals	Low	**Resolved**
[L-04]	ExecuteBatchBypass should be restricted to the governor address	Low	Acknowledged
[L-05]	Incorrect registry event on duplicate selector updates	Low	**Resolved**
[L-06]	Reward token removal without prior accrual loses user reward	Low	Acknowledged
[L-07]	Removing a reward token strand unaccounted balance in vault	Low	Acknowledged
[L-08]	Optimistic and standard proposals collide in shared ID space	Low	Acknowledged
[L-09]	Governor asserts on CLOCK_MODE causing gas exhaustion failure	Low	**Resolved**
[L-10]	OptimisticSelectorRegistry allows zero address governor	Low	**Resolved**

ID	Title	Severity	Status
[L-11]	Optimistic proposal state boundaries incorrect due to `>=` comparison	Low	Acknowledged

Medium findings

[M-01] Improper order of operations in `cancelLock` expose funds to theft

Severity

Impact: Medium

Likelihood: Medium

Description

Function `cancelLock()` does not follow the CEI pattern; it first makes the external call and then deletes the `locks[]` variable. If the execution reaches an attacker-controlled address during the external call, then it would be possible to perform the same operation multiple times and steal other users' funds.

```
        SafeERC20.forceApprove(targetToken, address(vault), lock.amount);
        vault.deposit(lock.amount, lock.user);

        emit LockCancelled(lockId);

        delete locks[lockId];
```

Recommendations: Move `delete locks[lockId]` to before the `SafeERC20.forceApprove` and `vault.deposit` calls to follow the Checks-Effects-Interactions pattern. Note: the vulnerability is not currently exploitable under the assumption of standard ERC20 tokens (no ERC777/transfer hooks), but would become actively exploitable if tokens with callbacks or an alternative vault implementation were introduced.

[M-02] CurrentAccountedNativeRewards uses current balance for reward

Severity

Impact: Low

Likelihood: High

Description

In function `_currentAccountedNativeRewards()`, when the code wants to calculate the rewards balance for the current timestamp instead of the last calculation time:

```
function _currentAccountedNativeRewards() internal view returns (uint256) {
    uint256 elapsed = block.timestamp - nativeRewardsLastPaid;
    uint256 rewardsBalance = IERC20(asset()).balanceOf(address(this)) - totalDeposited;

    return _calculateHandout(rewardsBalance, elapsed);
}
```

As a result of this, if some rewards are transferred to the contract, then the contract will distribute them for the past time too (It would be like those new reward tokens that were deposited from `nativeRewardsLastPaid` to the current time). For example:

1. There are 100 reward tokens in the contract, and each X contract distributes 10% of the rewards.
2. X seconds have passed since `nativeRewardsLastPaid`, and another 100 reward tokens are transferred to the contract.
3. Now, if `_currentAccountedNativeRewards` is called, the code would calculate the `rewardsBalance` as 200 and will distribute `200 * 10% = 20` tokens while the new 100 tokens are transferred right now, and the reward should have been `100 * 10% = 10`.

Recommendations

Either call poke more frequently to reduce the impact of the issue and accept the small distribution error or handle the native reward in the same way the contract handles the extra reward tokens by keeping track of `balanceLastKnown` and `balanceAccounted` and updating them and calculating `rewardsBalance` based on those snapshot values. Non-native reward tokens already implement the correct pattern: they track `balanceLastKnown` and `balanceAccounted` so that only the delta since the last snapshot is treated as new rewards. The native asset accounting should mirror this by introducing a `nativeBalanceLastKnown` variable (updated after each accrual in `_accrueRewards`), so that `rewardsBalance` is computed as `currentBalance - nativeBalanceLastKnown` rather than `currentBalance - totalDeposited`.

Low findings

[L-01] Eoa check can be bypassed

Description

Code performs this check `target.code.length != 0` to make sure that the target address of proposals is not an EOA account:

```
            require(
                target.code.length != 0 || proposal.calldatas[i].length == 0,
                IReserveOptimisticGovernor.InvalidCall(target, proposal.calldatas[i])
            );
```

The issue is that after the EIP-7702 upgrade, the value of `target.code.length` for EOA during the transaction can be higher than 0, and as a result, it would be possible to bypass this check.

[L-02] Mismatched rounding in ThrottleLib reduces effective proposal capacity

Description

When a proposal is consumed, the required charge is calculated using ceiling division (rounding up). However, when calculating the available proposals, the formula uses standard integer division (rounding down).

Consumption uses a rounding up cost per proposal:

```
    function consumeProposalCharge(ProposalThrottleStorage storage proposalThrottle, address
account) external {
        ...

>>>     throttle.currentCharge = charge - ((1e18 + proposalThrottle.capacity - 1) /
proposalThrottle.capacity); // roundup
        throttle.lastUpdated = block.timestamp;
    }
```

While availability uses rounding down:

```
    function _getProposalsAvailable(ProposalThrottleStorage storage proposalThrottle,
address account)
        private
        view
        returns (uint256 proposalsAvailable, uint256 charge)
    {
        ...
```

```
        proposalsAvailable = (proposalThrottle.capacity * charge) / 1e18; // round down
    }
```

So when `1e18` is not divisible by `capacity` (3, 6, 7, 9), the real capacity is reduced by one.

For example, with `capacity = 3`:

- Start: `charge = 1e18` → `available = 3`
- After 1 consume: `charge = 1e18 - ceil(1e18/3) = 333333333333333334` => `Remaining charge = 666666666666666666` => `proposalsAvailable = (3 × 666666666666666666) / 1e18 = 1.999999999999999998 = 1`
- After 2 consumes (same timestamp): `available = 0` (expected 1)

Proposers are unfairly throttled. In the example above, a user with a capacity of 3 can only submit 2 continuous proposals. They must wait for a while and the charge to be refilled before they can submit their 3rd authorized proposal.

[L-03] Uint256 overflow in reward calculations for tokens with high decimals

Description

According to the docs, `StakingVault` asset tokens can have up to 27 decimals and a maximum supply of 1e36. Under these conditions, the mathematical operations in `_accrueRewards` and `_accrueUser` are vulnerable to intermediate uint256 overflows.

In `_accrueRewards`, if decimals = 27 then `SCALAR * 10**decimals` equals `1e45`, the available room before hitting the type(uint256).max (~1.15e77) is only ~1.15e32. If the reward token also has 27 decimals, an overflow will occur if `tokensToHandout` exceeds just 115,000 tokens.

```
    function _accrueRewards(address _rewardToken) internal {
        ...
        if (tokensToHandout != 0) {
            // D18+decimals{reward/share} = D18 * {reward} * decimals / {share}
>>>         uint256 deltaIndex = (SCALAR * tokensToHandout * uint256(10 ** decimals())) /
totalSupply();

            // D18+decimals{reward/share} += D18+decimals{reward/share}
            rewardInfo.rewardIndex += deltaIndex;
            rewardInfo.balanceAccounted += tokensToHandout;
        }

        rewardInfo.payoutLastPaid = block.timestamp;
    }
```

In `_accrueUser`, if a user holds a large balance (e.g., 1e36 for a 27 decimal token) and `deltaIndex` is inflated due to high decimals (reward token decimal * SCALAR), the intermediate multiplication `balanceOf(_user) * deltaIndex` can exceed `type(uint256).max`, causing a revert.

```
    function _accrueUser(address _user, address _rewardToken) internal {
        if (_user == address(0)) {
            return;
        }

        RewardInfo memory rewardInfo = rewardTrackers[_rewardToken];
        UserRewardInfo storage userRewardTracker = userRewardTrackers[_rewardToken][_user];

        // D18+decimals{reward/share}
        uint256 deltaIndex = rewardInfo.rewardIndex - userRewardTracker.lastRewardIndex;

        if (deltaIndex != 0) {
            // Accumulate rewards by multiplying user tokens by index and adding on unclaimed
            // {reward} = {share} * D18+decimals{reward/share} / decimals / D18
>>>         uint256 supplierDelta = (balanceOf(_user) * deltaIndex) / uint256(10 **
decimals()) / SCALAR;

            // {reward} += {reward}
            userRewardTracker.accruedRewards += supplierDelta;
            userRewardTracker.lastRewardIndex = rewardInfo.rewardIndex;
        }
    }
```

If limits are hit, core operations (deposits, withdrawals, and reward claims) will consistently revert due to overflow, effectively locking user funds.

It is recommended to use `Math.mulDiv` to compute these values:

```
- uint256 deltaIndex = (SCALAR * tokensToHandout * uint256(10 ** decimals())) /
totalSupply();
+ uint256 deltaIndex = Math.mulDiv(tokensToHandout, SCALAR * (10 ** decimals()),
totalSupply());
```

```
- uint256 supplierDelta = (balanceOf(_user) * deltaIndex) / uint256(10 ** decimals()) /
SCALAR;
+ uint256 supplierDelta = Math.mulDiv(balanceOf(_user), deltaIndex, SCALAR * (10 **
decimals()));
```

[L-04] ExecuteBatchBypass should be restricted to the governor address

Description

executeBatchBypass is gated by `onlyRole(PROPOSER_ROLE)` and internally calls `executeBatch`, which requires the caller to also hold `EXECUTOR_ROLE`. In the current deployment, only `ReserveOptimisticGovernor` holds both roles, so the restriction behaves as intended.

However, both roles are standard `AccessControl` roles with no additional restrictions; they can be granted to arbitrary addresses through governance. If any address ever receives both `PROPOSER_ROLE` and `EXECUTOR_ROLE` in the future, it can call `executeBatchBypass` directly to execute arbitrary timelock operations with no delay, circumventing the security guarantees of the governance and timelock.

```
    function executeBatchBypass(
        address[] calldata targets,
        uint256[] calldata values,
        bytes[] calldata payloads,
        bytes32 predecessor,
        bytes32 salt
    ) public payable onlyRole(PROPOSER_ROLE) {
        bytes32 id = hashOperationBatch(targets, values, payloads, predecessor, salt);

        TimelockControllerStorage storage $ = _getTimelockControllerStorage();

        // mark Ready
        require($._timestamps[id] == 0, TimelockControllerOptimistic__OperationConflict());
        $._timestamps[id] = block.timestamp;

        // check caller has EXECUTOR_ROLE and execute
        executeBatch(targets, values, payloads, predecessor, salt);
    }
```

It is recommended to restrict `executeBatchBypass` to the governor address directly rather than relying on role membership:

```
function executeBatchBypass(...) public payable {
    require(msg.sender == governor, UnauthorizedCaller(msg.sender));
    ...
}
```

[L-05] Incorrect registry event on duplicate selector updates

Description

The _add and _remove functions in OptimisticSelectorRegistry emit SelectorsAdded and SelectorsRemoved using the full input selector array, even when some or all of those selectors were already present or already absent. Since EnumerableSet.add() and EnumerableSet.remove() silently return false when no state change occurs, the emitted events can claim that selectors were added or removed even though the underlying registry state did not change for those entries.

[L-06] Reward token removal without prior accrual loses user reward

Description

`removeRewardToken` removes a token from the active `rewardTokens` set without first accruing rewards up to the current timestamp. Additionally, `_rewardToken` is removed from the `rewardTokens` address set.

```
    function removeRewardToken(address _rewardToken) external onlyRole(DEFAULT_ADMIN_ROLE) {
        disallowedRewardTokens[_rewardToken] = true;

>>>     require(rewardTokens.remove(_rewardToken), Vault__RewardNotRegistered());

        emit RewardTokenRemoved(_rewardToken);
    }
```

Since `_accrueRewards` only iterates `rewardTokens.values`, once a token is removed, its global `rewardIndex` is frozen permanently. Any user whose `lastRewardIndex` is behind the final `rewardIndex` at removal time will have their delta dropped. Their `accruedRewards` for that token will never be updated.

The underlying token balance representing those unaccrued rewards becomes permanently stuck in the vault.

Furthermore, the `disallowedRewardTokens[_rewardToken] = true` flag set at removal time permanently prevents the token from ever being re-added via `addRewardToken`, making the reward lock irreversible — there is no recovery path for the stranded funds.

```
    function _accrueRewards(address _caller, address _receiver) internal {
>>>     address[] memory _rewardTokens = rewardTokens.values();
        uint256 _rewardTokensLength = _rewardTokens.length;

        for (uint256 i; i < _rewardTokensLength; i++) {
            address rewardToken = _rewardTokens[i];

>>>         _accrueRewards(rewardToken); // not call for removed token
>>>         _accrueUser(_receiver, rewardToken); // not call for removed token

            if (_receiver != _caller) {
                _accrueUser(_caller, rewardToken);
            }
        }

        /**

         * Native asset() rewards are special cased
         */
        totalDeposited += _currentAccountedNativeRewards();
        nativeRewardsLastPaid = block.timestamp;
    }
```

Consider a scenario:

- User stakes and never touches the vault again.
- Rewards accrue over time (global `rewardIndex` grows).
- Governance removes the reward token before this user ever calls `claimRewards` / `poke` / `transfers`.
- This user's `accruedRewards` for that token may still be `0`, and after removal it will never be updated, so they cannot claim what they have "earned" up to removal time.

Recommendations

- In `removeRewardToken`, accrue the token globally before removing it: `_accrueRewards(token)`
- Note: `claimRewards` does not gate on `rewardTokens` set membership, so callers can technically pass a removed token address. However, it relies on the `accrueRewards` modifier to sync `lastRewardIndex` — and that modifier iterates only active `rewardTokens`, silently skipping removed tokens. This is why an inline `_accrueUser(msg.sender, token)` call is required inside `claimRewards` for any token not in the active set.
- In `claimRewards`, call `_accrueUser(msg.sender, _rewardToken)` before reading `accruedRewards`, so users who interact after removal can still collect their share earned up to the removal block.

[L-07] Removing a reward token strand unaccounted balance in vault

Description

When `removeRewardToken()` is called, the token is immediately removed from the `rewardTokens` EnumerableSet without performing a final global accrual:

```
function removeRewardToken(address _rewardToken) external onlyRole(DEFAULT_ADMIN_ROLE) {
    disallowedRewardTokens[_rewardToken] = true;
    require(rewardTokens.remove(_rewardToken), Vault__RewardNotRegistered());
    emit RewardTokenRemoved(_rewardToken);
}
```

The global `_accrueRewards(address _rewardToken)` is only called for tokens in the `rewardTokens` set, which is iterated inside the `_accrueRewards(address _caller, address _receiver)` loop. Once a token is removed from the set, this function is never called for it again.

This means any reward tokens that arrived at the vault between the last global accrual and the moment of removal are **never reflected in** `rewardIndex`. The `unaccountedBalance` that would have been converted into `tokensToHandout` and added to `rewardInfo.balanceAccounted` is simply abandoned. Those tokens sit in the vault contract's balance with no accounting path to ever distribute or recover them.

Since the token is also permanently blacklisted via `disallowedRewardTokens[_rewardToken] = true`, it cannot be re-added to trigger a final accrual.

Recommendation

Perform a final global accrual before removing the token:

```
function removeRewardToken(address _rewardToken) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _accrueRewards(_rewardToken); // final index update
    disallowedRewardTokens[_rewardToken] = true;
    require(rewardTokens.remove(_rewardToken), Vault__RewardNotRegistered());
    emit RewardTokenRemoved(_rewardToken);
}
```

[L-08] Optimistic and standard proposals collide in shared ID space

Description

In `ReserveOptimisticGovernor`, both `proposeOptimistic()` and `propose()` compute the proposal ID via `getProposalId(targets, values, calldatas, descriptionHash)`. If the same `(targets, values, calldatas, description)` tuple is used for both paths, they produce the same `proposalId`. Since `ProposalLib` checks that `voteStart != 0` for existing proposals and reverts on collision, whichever proposal type is created first blocks the other.

A concrete attack:

1. An attacker who meets the `proposalThreshold` for standard proposals watches the mempool for `proposeOptimistic()` calls.
2. The attacker front-runs with `propose()` using the same parameters.
3. The `proposeOptimistic()` call reverts because the proposal ID already exists.

This works in reverse too: an optimistic proposer can block a standard proposal. The attack is especially problematic because standard proposals are permissionless (anyone meeting the threshold can propose), while optimistic proposals require `OPTIMISTIC_PROPOSER_ROLE`. A standard proposer can thus grief optimistic proposers by front-running their proposals.

Recommendations

Differentiate proposal ID derivation between optimistic and standard proposals, for example, by including a type prefix or flag in the hash. Alternatively, add a salt or nonce that distinguishes the proposal type within the hash computation. The simplest fix is to include a boolean `isOptimistic` flag in the `getProposalId` hash.

[L-09] Governor asserts on CLOCK_MODE causing gas exhaustion failure

Description

In `contracts/governance/ReserveOptimisticGovernor.sol`, `initialize()` begins with:

```
assert(keccak256(bytes(IERC5805(_token).CLOCK_MODE())) == keccak256("mode=timestamp"));
```

If `_token` is not a compliant IERC5805 timestamp-clock token (e.g., using `deployWithExistingStakingVault` with a non-conforming contract, or any external deployment that passes an unexpected token), this triggers an `assert` failure which compiles to an invalid opcode and consumes all remaining gas rather than producing a normal revert with a reason.

This is reachable directly through the `initialize(...)` entry point on any freshly deployed governor proxy (or any deployment flow that passes a bad token), and it makes debugging and safe failure handling materially worse than a standard `require`. While it does not enable privilege escalation, it increases the operational DoS risk and makes misconfiguration failures more costly and opaque.

Recommendations

Replace the `assert(...)` with a `require(...)` that reverts cleanly with a custom error (e.g., `InvalidClockMode()`), and optionally pre-check `_token.code.length != 0` to fail early and clearly. This preserves the invariant while avoiding the all-gas-consumption behavior of `assert` for user-supplied or externally integrated tokens.

[L-10] OptimisticSelectorRegistry allows zero address governor

Description

In `OptimisticSelectorRegistry.sol`, the `initialize` function sets `governor = ReserveOptimisticGovernor(payable(_governor))` without checking that `_governor` is not equal to `address(0)`.

If `_governor` is the zero address, all subsequent calls through the `onlyTimelock` modifier will check `msg.sender == governor.timelock()`, which will revert (since `address(0)` has no code). Additionally, the `_add` function calls `governor.timelock()` and `governor.token()` to get blocked targets, which would also revert.

During the Deployer flow, the registry is initialized with a valid governor address (Step 2.4 in the deployment). The risk arises if the registry is deployed and initialized outside the standard Deployer flow (e.g., a custom deployment). A zero governor would render the registry permanently non-functional since the `initializer` modifier prevents re-initialization.

Recommendations

Add `require(_governor != address(0))` at the beginning of the `initialize` function to prevent accidental initialization with a zero address. This is a defense-in-depth measure.

[L-11] Optimistic proposal state boundaries incorrect due to `>=` comparison

Description

In `ReserveOptimisticGovernor.state()` for optimistic proposals, pending/active are determined by `if (snapshot >= block.timestamp) return Pending;` and active/succeeded by `if (deadline >= block.timestamp) return Active;`.

These comparisons make the proposal remain `Pending` at `block.timestamp == voteStart` (snapshot) and remain `Active` at `block.timestamp == voteStart + voteDuration` (deadline), meaning the veto voting window effectively starts one second after `voteStart` and ends one second after `deadline` compared to the more typical strict inequality model.

This is observable from the permissionless view entry point `state(proposalId)` and affects when `execute()` becomes possible since OZ Governor's execution path gates on proposal state. While the numerical discrepancy is small, it can surprise off-chain monitors/keepers that schedule veto participation or execution at exact boundary times, and it is inconsistent with common Governor semantics (active when `now >= voteStart` and `now < deadline`).

It does not directly enable an attacker to bypass veto, but it can create edge-case timing confusion during high contention moments.

Recommendations

Align boundary conditions with standard Governor semantics by using strict comparisons:

```
if (block.timestamp < snapshot) return ProposalState.Pending;
else if (block.timestamp < deadline) return ProposalState.Active;
else return ProposalState.Succeeded;
```

Add tests that check the state at exact `voteStart` and exact `deadline` timestamps to ensure the intended boundaries match off-chain monitoring expectations.